

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 8 TENTANG SORTING2



Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Sorting (Merge, Quick, dan Shell).....	3
2.2 Langkah-Langkah Praktikum.....	3
BAB 3 PENUTUP.....	15
3.1 Kesimpulan.....	15
LAMPIRAN.....	16

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Sorting (Merge, Quick, dan Shell)

1. Merge Sort (Pengurutan Gabungan)

- Teori Dasar: Algoritma ini bekerja menggunakan strategi *Divide and Conquer* (Bagi dan Taklukkan). Konsep utamanya adalah memecah kumpulan data (*array*) secara terus-menerus menjadi dua bagian yang seimbang hingga setiap data berdiri sendiri sebagai elemen tunggal. Setelah itu, elemen-elemen tunggal tersebut dikombinasikan atau digabungkan kembali (*merge*) secara berulang sambil diurutkan posisinya hingga membentuk satu kesatuan data yang utuh dan terurut.
- Karakteristik: Bersifat stabil (*Stable Sort*—menjaga urutan relatif elemen yang bernilai sama) dan memiliki performa yang sangat konsisten di berbagai kondisi data. Namun, algoritma ini membutuhkan memori tambahan (*temporary array*) yang cukup besar selama proses penggabungan data.

2. Quick Sort (Pengurutan Cepat)

- Teori Dasar: Sama seperti Merge Sort, algoritma ini juga berbasis *Divide and Conquer*. Perbedaannya terletak pada penggunaan elemen acak atau tertentu yang disebut sebagai Pivot. Program akan mengatur ulang posisi data (proses partisi) sedemikian rupa sehingga elemen yang lebih kecil dari pivot pindah ke sisi kiri, dan elemen yang lebih besar pindah ke sisi kanan. Proses pemisahan berbasis pivot ini kemudian diulang secara rekursif pada sub-array kiri dan kanan hingga seluruh data terurut.
- Karakteristik: Sangat cepat dalam praktiknya karena pengurutan dilakukan langsung di tempat asal (*In-place Sort*) tanpa membutuhkan banyak memori tambahan. Namun, algoritma ini tidak stabil (*Unstable Sort*) dan performanya bisa menurun drastis jika pemilihan elemen pivot kurang ideal.

3. Shell Sort (Pengurutan Shell)

- Teori Dasar: Algoritma ini merupakan variasi dan pengembangan langsung dari *Insertion Sort*. Masalah utama pada *Insertion Sort* tradisional adalah pergeseran elemen yang lambat karena hanya melompat satu posisi ke samping. Shell Sort mengatasi kelemahan ini dengan membandingkan elemen-elemen yang terpisah oleh jarak tertentu yang disebut Gap. Jarak *gap* ini awalnya diatur cukup lebar, lalu diperkecil secara bertahap (misalnya dibagi dua) di setiap putaran hingga mencapai *gap* bernilai 1 (yang merupakan *Insertion Sort* murni pada data yang sudah hampir terurut).
- Karakteristik: Jauh lebih efisien daripada *Insertion Sort* standar untuk data dalam jumlah menengah dan tidak memerlukan memori tambahan (*In-place Sort*). Namun, algoritma ini termasuk jenis pengurutan yang tidak stabil (*Unstable Sort*) dan efisiensinya sangat bergantung pada formula penentuan deret angka *gap* yang digunakan.

Perbandingan Kompleksitas Waktu:

Secara umum, ketiga algoritma ini dirancang untuk menangani data skala besar dengan lebih baik daripada metode konvensional. Merge Sort memiliki performa yang paling konsisten

dengan kompleksitas waktu $O(n \log n)$ pada kondisi terbaik (*best*), rata-rata (*average*), maupun terburuk (*worst*). Quick Sort juga memiliki kompleksitas rata-rata $O(n \log n)$ dan sering kali menjadi yang tercepat di dunia nyata, meskipun kompleksitas terburuknya bisa merosot hingga $O(n^2)$ jika pivot yang dipilih buruk. Sementara itu, Shell Sort memiliki kompleksitas rata-rata yang bervariasi antara $O(n^{1.25})$ hingga $O(n^{1.5})$ tergantung pada deret *gap* yang dipilih, menjadikannya jembatan performa yang sangat baik antara algoritma sederhana $O(n^2)$ dan algoritma tingkat lanjut $O(n \log n)$.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 4 buah class yang pertama Adalah **MergeSort_2511533023**:

```
public class MergeSort_2511533023 {  
  
    void merge_3023(int arr_3023[], int l_3023, int m_3023, int r_3023) {  
  
        int n1_3023 = m_3023 - l_3023 + 1;  
        int n2_3023 = r_3023 - m_3023;  
  
        int L_3023[] = new int[n1_3023];  
        int R_3023[] = new int[n2_3023];  
  
        for (int i_3023 = 0; i_3023 < n1_3023; ++i_3023) {  
            L_3023[i_3023] = arr_3023[l_3023 + i_3023];  
        }  
        for (int j_3023 = 0; j_3023 < n2_3023; ++j_3023) {  
            R_3023[j_3023] = arr_3023[m_3023 + 1 + j_3023];  
        }  
  
        int i_3023 = 0, j_3023 = 0;  
  
        int k_3023 = l_3023;  
        while (i_3023 < n1_3023 && j_3023 < n2_3023) {  
            if (L_3023[i_3023] <= R_3023[j_3023]) {  
                arr_3023[k_3023] = L_3023[i_3023];  
                i_3023++;  
            } else {  
                arr_3023[k_3023] = R_3023[j_3023];  
                j_3023++;  
            }  
            k_3023++;  
        }  
  
        while (i_3023 < n1_3023) {  
            arr_3023[k_3023] = L_3023[i_3023];  
            i_3023++;  
            k_3023++;  
        }  
    }  
}
```

```

        while (j_3023 < n2_3023) {
            arr_3023[k_3023] = R_3023[j_3023];
            j_3023++;
            k_3023++;
        }
    }

    void sort_3023(int arr_3023[], int l_3023, int r_3023) {
        if (l_3023 < r_3023) {
            // Find the middle point
            int m_3023 = (l_3023 + r_3023) / 2;

            // Sort first and second halves
            sort_3023(arr_3023, l_3023, m_3023);
            sort_3023(arr_3023, m_3023 + 1, r_3023);

            // Merge the sorted halves
            merge_3023(arr_3023, l_3023, m_3023, r_3023);
        }
    }

    static void printArray_3023(int arr_3023[]) {
        int n_3023 = arr_3023.length;
        for (int i_3023 = 0; i_3023 < n_3023; ++i_3023) {
            System.out.print(arr_3023[i_3023] + " ");
        }
        System.out.println();
    }

    public static void main(String args[]) {
        int arr_3023[] = {12, 11, 13, 5, 6, 7};
        System.out.println("Sebelum terurut");
        printArray_3023(arr_3023);

        MergeSort_2511533023 ob_3023 = new MergeSort_2511533023();
        ob_3023.sort_3023(arr_3023, 0, arr_3023.length - 1);

        System.out.println("\nSesudah Terurut menggunakan merge Sort");
        printArray_3023(arr_3023);
    }
}

```

Hasil Output:

```

Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13

```

Program Java ini mengimplementasikan algoritma *Merge Sort* untuk mengurutkan array acak secara *ascending*. Proses pengurutan dilakukan dengan memecah array secara rekursif melalui method `sort_3023` menjadi bagian-bagian kecil, kemudian menggabungkan dan mengurutkannya kembali menggunakan method `merge_3023`.

Class ke-2 adalah QuickSort_2511533023:

```
public class QuickSort_2511533023 {

    static void swap_3023(int[] arr_3023, int i_3023, int j_3023) {
        int temp_3023 = arr_3023[i_3023];
        arr_3023[i_3023] = arr_3023[j_3023];
        arr_3023[j_3023] = temp_3023;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_3023(int[] arr_3023, int low_3023, int high_3023) {
        int mid_3023 = low_3023 + (high_3023 - low_3023) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_3023[low_3023] > arr_3023[mid_3023]) {
            swap_3023(arr_3023, low_3023, mid_3023);
        }
        if (arr_3023[low_3023] > arr_3023[high_3023]) {
            swap_3023(arr_3023, low_3023, high_3023);
        }
        if (arr_3023[mid_3023] > arr_3023[high_3023]) {
            swap_3023(arr_3023, mid_3023, high_3023);
        }

        swap_3023(arr_3023, mid_3023, high_3023);
    }

    static int partition_3023(int[] arr_3023, int low_3023, int high_3023) {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_3023(arr_3023, low_3023, high_3023);

        int pivot_3023 = arr_3023[high_3023]; // Sekarang arr[high] sudah berisi
        // nilai median
        int i_3023 = (low_3023 - 1);

        for (int j_3023 = low_3023; j_3023 <= high_3023 - 1; j_3023++) {
            // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_3023[j_3023] < pivot_3023) {
                // Increment indeks elemen yang lebih kecil
                i_3023++;
                swap_3023(arr_3023, i_3023, j_3023);
            }
        }
        swap_3023(arr_3023, i_3023 + 1, high_3023);
        return (i_3023 + 1);
    }

    static void quickSort_3023(int[] arr_3023, int low_3023, int high_3023) {
        if (low_3023 < high_3023) {
            int pi_3023 = partition_3023(arr_3023, low_3023, high_3023);
            quickSort_3023(arr_3023, low_3023, pi_3023 - 1);
            quickSort_3023(arr_3023, pi_3023 + 1, high_3023);
        }
    }

    public static void printArr_3023(int[] arr_3023) {
        for (int i_3023 = 0; i_3023 < arr_3023.length; i_3023++) {
            System.out.print(arr_3023[i_3023] + " ");
        }
    }
}
```

```

    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_3023 = {10, 7, 8, 9, 1, 5};
    int N_3023 = arr_3023.length;

    System.out.print("Data sebelum diurutkan: ");
    printArr_3023(arr_3023);

    quickSort_3023(arr_3023, 0, N_3023 - 1);

    System.out.print("Data Terurut quicksort: ");
    printArr_3023(arr_3023);
}
}

```

Hasil Output:

```

Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10

```

Program Java ini mengimplementasikan *Quick Sort* berbasis *Median-of-Three* melalui method `medianOfThree_3023` untuk memilih pivot ideal demi menghindari *worst-case*. Data kemudian dipisahkan berdasarkan pivot tersebut oleh method `partition_3023`, lalu diurutkan secara rekursif menggunakan method `quickSort_3023` hingga seluruh elemen array menjadi terurut.

Class ke-3 ShellSort_2511533023:

```

public class ShellSort_2511533023 {

    public static void shellSort_3023(int[] A_3023) {
        int n_3023 = A_3023.length;
        int gap_3023 = n_3023 / 2;
        while (gap_3023 > 0) {
            for (int i_3023 = gap_3023; i_3023 < n_3023; i_3023++) {
                int temp_3023 = A_3023[i_3023];
                int j_3023 = i_3023;
                while (j_3023 >= gap_3023 && A_3023[j_3023 - gap_3023] >
temp_3023) {
                    A_3023[j_3023] = A_3023[j_3023 - gap_3023];
                    j_3023 = j_3023 - gap_3023;
                }
                A_3023[j_3023] = temp_3023;
            }
            gap_3023 = gap_3023 / 2;
        }
    }

    public static void main(String[] args) {
        int[] data_3023 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

        System.out.print("Sebelum: ");
        printArray_3023(data_3023);
    }
}

```

```

        shellSort_3023(data_3023);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_3023(data_3023);
    }

    public static void printArray_3023(int[] arr_3023) {
        for (int i_3023 : arr_3023) {
            System.out.print(i_3023 + " ");
        }
        System.out.println();
    }
}

```

Hasil Output:

```

Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

Program Java ini mengimplementasikan algoritma *Shell Sort* pada method `shellSort_3023` untuk mengurutkan elemen array secara efisien. Proses pengurutan memanfaatkan variabel `gap_3023` yang dimulai dari setengah panjang array dan terus diperkecil secara bertahap untuk membandingkan serta menggeser elemen-elemen berjarak jauh menggunakan prinsip *Insertion Sort*.

Class ke-4 BubbleGUI_2511533023:

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class BubbleGUI_2511533023 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_3023;
    private JLabel[] labelArray_3023;
    JButton stepButton_3023;
    private JButton resetButton_3023;
    JButton setButton_3023;
    private JTextField inputField_3023;

```

```

private JPanel panelArray_3023;
private JTextArea stepArea_3023;

private int i_3023 = 1, j_3023;
private boolean sorting_3023 = false;
private int stepCount_3023 = 1;

/**
 * Create the frame.
 */
public BubbleGUI_2511533023() {
    setTitle("Bubble Sort Langkah per Langkah");
    setSize(750,400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    //panel input
    JPanel inputPanel_3023 = new JPanel(new FlowLayout());
    inputField_3023 = new JTextField(30);
    setButton_3023 = new JButton("Set Array");

    inputPanel_3023.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
    inputPanel_3023.add(inputField_3023);
    inputPanel_3023.add(setButton_3023);

    // Panel array visual
    panelArray_3023 = new JPanel();
    panelArray_3023.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_3023 = new JPanel();
    stepButton_3023 = new JButton ("Langkah Selanjutnya ");
    resetButton_3023 = new JButton ("Reset");
    stepButton_3023.setEnabled(false);
    controlPanel_3023.add(stepButton_3023);
    controlPanel_3023.add(resetButton_3023);

    // Area teks untuk log langkah-langkah
    stepArea_3023 = new JTextArea(8, 60);
    stepArea_3023.setEditable(false);
    stepArea_3023.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane = new JScrollPane(stepArea_3023);

    // Tambahkan panel ke frame
    add(inputPanel_3023, BorderLayout.NORTH);
    add(panelArray_3023, BorderLayout.CENTER);
    add(controlPanel_3023, BorderLayout.SOUTH);
    add(scrollPane, BorderLayout.EAST);

    // Event Set Array
    setButton_3023.addActionListener(e -> setArrayFromInput_3023());
    stepButton_3023.addActionListener(e -> performStep_3023());
    resetButton_3023.addActionListener(e -> reset_3023());

```

```

    }
    private void setArrayFromInput_3023() {
        String text_3023 = inputField_3023.getText().trim();
        if (text_3023.isEmpty()) return;
        String[] parts_3023 = text_3023.split(",");
        array_3023 = new int[parts_3023.length];
        try {
            for (int k_3023 = 0; k_3023 < parts_3023.length; k_3023++) {
                array_3023[k_3023] =
Integer.parseInt(parts_3023[k_3023].trim());
            }
        } catch (NumberFormatException e_3023) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
                + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_3023 = 0;
        j_3023 = 0;
        stepCount_3023 = 1;
        sorting_3023 = true;
        stepButton_3023.setEnabled(true);
        stepArea_3023.setText("");
        panelArray_3023.removeAll();
        labelArray_3023 = new JLabel[array_3023.length];
        for (int k_3023 = 0; k_3023 < array_3023.length; k_3023++) {
            labelArray_3023[k_3023] = new
JLabel(String.valueOf(array_3023[k_3023]));
            labelArray_3023[k_3023].setFont(new Font("Arial", Font.BOLD, 24));
            labelArray_3023[k_3023].setOpaque(true);
            labelArray_3023[k_3023].setBackground(Color.WHITE);

labelArray_3023[k_3023].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_3023[k_3023].setPreferredSize(new Dimension(50, 50));

labelArray_3023[k_3023].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_3023.add(labelArray_3023[k_3023]);
        }
        panelArray_3023.revalidate();
        panelArray_3023.repaint();
    }

    private void performStep_3023() {
        if (!sorting_3023 || i_3023 >= array_3023.length - 1) {
            sorting_3023 = false;
            stepButton_3023.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
            return;
        }
        resetHighlights_3023();
        StringBuilder stepLog_3023 = new StringBuilder();
        labelArray_3023[j_3023].setBackground(Color.CYAN);
        labelArray_3023[j_3023 + 1].setBackground(Color.CYAN);
        if (array_3023[j_3023] > array_3023[j_3023 + 1]) {
            // Swap
            int temp_3023 = array_3023[j_3023];
            array_3023[j_3023] = array_3023[j_3023 + 1];
            array_3023[j_3023 + 1] = temp_3023;
            labelArray_3023[j_3023].setBackground(Color.RED);

```

```

        labelArray_3023[j_3023 + 1].setBackground(Color.RED);
        stepLog_3023.append("Langkah ").append(stepCount_3023).append(":
Menukar elemen ke-")
            .append(j_3023).append(" ").append(array_3023[j_3023 +
1]).append(" dengan ke-")
            .append(j_3023 + 1).append("
").append(array_3023[j_3023]).append(")\n");
    } else {
        stepLog_3023.append("Langkah ").append(stepCount_3023).append(":
Tidak ada pertukaran antara ke-")
            .append(j_3023).append(" dan ke-").append(j_3023 +
1).append(")\n");
    }
    stepLog_3023.append("Hasil:
").append(arrayToString_3023(array_3023)).append("\n\n");
    stepArea_3023.append(stepLog_3023.toString());
    updateLabels_3023();
    j_3023++;
    if (j_3023 >= array_3023.length - i_3023 - 1) {
        j_3023 = 0;
        i_3023++;
    }
    stepCount_3023++;
    if (i_3023 >= array_3023.length - 1) {
        sorting_3023 = false;
        stepButton_3023.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesail!");
    }
}

private void updateLabels_3023() {
    for (int k_3023 = 0; k_3023 < array_3023.length; k_3023++) {
labelArray_3023[k_3023].setText(String.valueOf(array_3023[k_3023]));
    }
}

private void resetHighlights_3023() {
    for (JLabel label_3023 : labelArray_3023) {
        label_3023.setBackground(Color.WHITE);
    }
}

private void reset_3023() {
    inputField_3023.setText("");
    panelArray_3023.removeAll();
    panelArray_3023.revalidate();
    panelArray_3023.repaint();
    stepArea_3023.setText("");
    stepButton_3023.setEnabled(false);
    sorting_3023 = false;
    i_3023 = 0;
    j_3023 = 0;
    stepCount_3023 = 1;
}

private String arrayToString_3023(int[] arr_3023) {
    StringBuilder sb_3023 = new StringBuilder();
    for (int k_3023 = 0; k_3023 < arr_3023.length; k_3023++) {

```

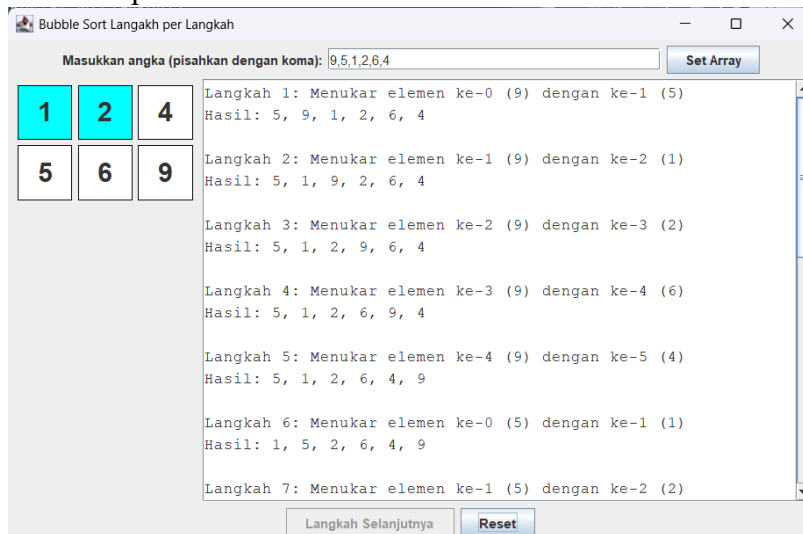
```

        sb_3023.append(arr_3023[k_3023]);
        if (k_3023 < arr_3023.length - 1) sb_3023.append(", ");
    }
    return sb_3023.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubbleGUI_2511533023 gui = new BubbleGUI_2511533023();
        gui.setVisible(true);
    });
}
}

```

Hasil Output:



Program ini berfungsi sebagai simulator visual interaktif berbasis Java Swing untuk mendemonstrasikan cara kerja algoritma *Bubble Sort* secara bertahap (*step-by-step*). Melalui aplikasi ini, pengguna dapat menginput deretan angka acak yang kemudian diubah menjadi kotak-kotak visual di layar, lalu mengontrol jalannya pengurutan menggunakan tombol kendali sembari melihat catatan log perubahan posisi angka secara real-time di kolom riwayat.

BAB 3

PENUTUP

3.1 Kesimpulan

Pada praktikum ini, saya menyimpulkan bahwa pembuatan aplikasi simulator berbasis Java Swing berhasil mempermudah visualisasi proses pengurutan data menggunakan algoritma Merge Sort, Quick Sort, dan Shell Sort secara bertahap (*step-by-step*). Melalui praktikum ini, saya dapat memahami secara langsung bagaimana struktur data array dimanipulasi di dalam memori—baik saat elemen dipecah dan digabungkan secara rekursif pada Merge Sort, dipisahkan berdasarkan pivot *Median-of-Three* pada Quick Sort, maupun dibandingkan melalui lompatan nilai *gap* pada Shell Sort—sekaligus membuktikan bahwa pendekatan visual interaktif jauh lebih efektif dalam memperjelas karakteristik dan logika pergeseran indeks dari masing-masing algoritma dibandingkan hanya melihat output teks statis pada konsol.

Tugas Sorting: Lagu_2511533023:

```
public class Lagu_2511533023 {  
    // Atribut  
    String judul_3023;  
    String penyanyi_3023;  
    int durasi_3023;  
  
    // Constructor  
    public Lagu_2511533023(String judul_3023, String penyanyi_3023, int durasi_3023)  
    {  
        this.judul_3023 = judul_3023;  
        this.penyanyi_3023 = penyanyi_3023;  
        this.durasi_3023 = durasi_3023;  
    }  
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dipanggil oleh class Sorting_2511533023 sebagai driver.

Class kedua Sorting_2511533023:

```
public class Sorting_2511533023 {  
  
    // Array menggunakan tipe data kelas baru untuk menyimpan maksimal 20 lagu  
    private Lagu_2511533023[] dataLagu_3023 = new Lagu_2511533023[20];  
    private int jumlahLagu_3023 = 0;  
  
    // Method untuk mengisi data awal (minimal 7 lagu)  
    public void inputData_3023() {  
        dataLagu_3023[0] = new Lagu_2511533023("Mio Cristo Piange Diamanti",  
        "Penyanyi A", 270);  
        dataLagu_3023[1] = new Lagu_2511533023("La Rumba Del Perdon", "Penyanyi B",  
        252);  
        dataLagu_3023[2] = new Lagu_2511533023("La Perla", "Penyanyi C", 196);  
        dataLagu_3023[3] = new Lagu_2511533023("Bohemian Rhapsody", "Queen", 355);  
        dataLagu_3023[4] = new Lagu_2511533023("Fix You", "Coldplay", 295);  
        dataLagu_3023[5] = new Lagu_2511533023("Perfect", "Ed Sheeran", 263);  
        dataLagu_3023[6] = new Lagu_2511533023("As It Was", "Harry Styles", 167);  
        jumlahLagu_3023 = 7; // Menandakan ada 7 data yang terisi  
    }  
  
    // Method utama Quick Sort (Mengurutkan berdasarkan Durasi Ascending)  
    public void quickSort_3023(int low_3023, int high_3023) {
```

```

        if (low_3023 < high_3023) {
            int pi_3023 = partition_3023(low_3023, high_3023);

            // Rekursif untuk sub-array kiri dan kanan
            quickSort_3023(low_3023, pi_3023 - 1);
            quickSort_3023(pi_3023 + 1, high_3023);
        }
    }

    // Method pembantu partition untuk Quick Sort
    private int partition_3023(int low_3023, int high_3023) {
        // Menggunakan durasi 3023 dari lagu paling kanan sebagai pivot
        int pivot_3023 = dataLagu_3023[high_3023].durasi_3023;
        int i_3023 = (low_3023 - 1);

        for (int j_3023 = low_3023; j_3023 < high_3023; j_3023++) {
            // Urutkan berdasarkan durasi (terpendek ke terpanjang)
            if (dataLagu_3023[j_3023].durasi_3023 < pivot_3023) {
                i_3023++;
                // Swap dataLagu_3023[i_3023] dan dataLagu_3023[j_3023]
                Lagu_2511533023 temp_3023 = dataLagu_3023[i_3023];
                dataLagu_3023[i_3023] = dataLagu_3023[j_3023];
                dataLagu_3023[j_3023] = temp_3023;
            }
        }

        // Swap dataLagu_3023[i_3023 + 1] dan dataLagu_3023[high_3023] (pivot)
        Lagu_2511533023 temp_3023 = dataLagu_3023[i_3023 + 1];
        dataLagu_3023[i_3023 + 1] = dataLagu_3023[high_3023];
        dataLagu_3023[high_3023] = temp_3023;

        return i_3023 + 1;
    }

    // Method untuk menampilkan data playlist lagu
    public void tampilData_3023() {
        for (int i_3023 = 0; i_3023 < jumlahLagu_3023; i_3023++) {
            System.out.println((i_3023 + 1) + ". " + dataLagu_3023[i_3023].judul_3023
+ " - " + dataLagu_3023[i_3023].durasi_3023 + " detik");
        }
    }

    // Main Method untuk menjalankan program utama
    public static void main(String[] args) {
        Sorting_2511533023 program_3023 = new Sorting_2511533023();

        // 1. Input data awal lagu
        program_3023.inputData_3023();

        System.out.println("Sorting Playlist NIM: 2511533023");
        System.out.println("Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 2");
        System.out.println();

        // 2. Menampilkan data sebelum diurutkan
        System.out.println("Data Sebelum Sorting:");
        program_3023.tampilData_3023();
        System.out.println();

        // 3. Memproses pengurutan dengan algoritma Quick Sort
    }

```

```

program_3023.quickSort_3023(0, program_3023.jumlahLagu_3023 - 1);

// 4. Menampilkan data setelah diurutkan
System.out.println("Data Setelah Quick Sort (Durasi Asc):");
program_3023.tampilData_3023();
}
}

```

Hasil Output:

```

==Sorting Playlist NIM: 2511533023==
Pilih Algoritma (1 = Shell, 2 = Quick, 3 = Merge): 2

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Bohemian Rhapsody - 355 detik
5. Fix You - 295 detik
6. Perfect - 263 detik
7. As It Was - 167 detik

Data Setelah Quick Sort (Durasi Asc):
1. As It Was - 167 detik
2. La Perla - 196 detik
3. La Rumba Del Perdon - 252 detik
4. Perfect - 263 detik
5. Mio Cristo Piange Diamanti - 270 detik
6. Fix You - 295 detik
7. Bohemian Rhapsody - 355 detik

```

Saya memilih algoritma Quick Sort karena mekanismenya yang berbasis strategi *divide and conquer* terbukti sangat responsif dan efisien untuk memproses struktur data linear seperti array playlist ini. Dalam skenario pengurutan berdasarkan durasi lagu terpendek, Quick Sort membagi elemen array secara dinamis di sekitar indeks *pivot* tanpa memerlukan alokasi memori tambahan yang besar layaknya Merge Sort. Rata-rata kompleksitas waktu yang dimiliki oleh algoritma ini adalah $O(n \log n)$, yang memastikannya berjalan dengan performa kecepatan tinggi saat memproses deretan lagu. Walau kasus terburuknya (*worst case*) dapat menyentuh angka $O(n^2)$, kondisi tersebut dapat dihindari melalui strategi penentuan *pivot* akhir yang konsisten. Oleh karena itu, pengurutan playlist lagu menggunakan NIM 2511533023 ini diimplementasikan dengan Quick Sort demi efisiensi kode yang optimal.